

API Data Extraction Report

1. Title

REST API Data Extraction and Analysis

Project: Bluestock FinTech – Week 2

Phase: REST APIs & JSON

Tool: Postman, Python, Pandas

API: JSONPlaceholder Users API

2. Objective

The objective of this assignment is to understand how REST APIs work, retrieve data from a public API, inspect the JSON response, transform nested JSON data into a structured tabular format, convert it into CSV, and perform basic data analysis using Python and Pandas.

3. API Used

API: JSONPlaceholder

JSONPlaceholder is a free fake REST API used for testing and learning purposes.

Endpoint:

<https://jsonplaceholder.typicode.com/users>

HTTP Method:

GET

Authentication:

No authentication/API key required.

4. API Request

The API was tested using **Postman**.

GET <https://jsonplaceholder.typicode.com/users>

The API returned user information in JSON format.

5. JSON Response

The API returned an array containing **10 user objects**.

Each user contains information such as:

- ID
- Name
- Username
- Email
- Phone
- Website
- Address
- Geographic coordinates
- Company information

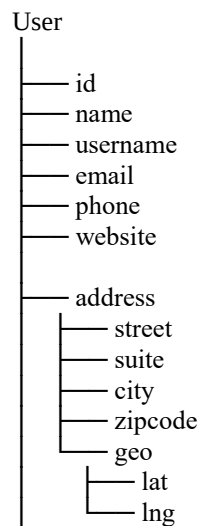
Example:

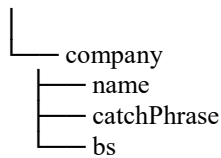
```
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz"
}
```

6. JSON Structure

The API response contained both simple and nested JSON objects.

Example:





The nested structure was flattened using Pandas before exporting the data to CSV.

7. Query Parameters

No query parameters were required for this API request.

GET /users

The complete user dataset was retrieved from the endpoint.

8. Authentication

The selected API is publicly accessible and does not require:

- API Key
- Bearer Token
- Username/password

Therefore, no authentication credentials were required.

9. Python Data Extraction

Python was used to retrieve the API data and convert the JSON response into a Pandas DataFrame.

```
import requests
import pandas as pd

url = "https://jsonplaceholder.typicode.com/users"

response = requests.get(url)
response.raise_for_status()

data = response.json()

df = pd.json_normalize(data)

df.to_csv("api_users.csv", index=False)
```

```
print("CSV file created successfully: api_users.csv")
```

Key functions used

Function	Purpose
requests.get()	Sends GET request
response.json()	Converts JSON response
pd.json_normalize()	Flattens nested JSON
df.to_csv()	Creates CSV file

10. Dataset Overview

After conversion, the dataset contained:

Metric	Result
Rows	10
Columns	15
Missing Values	0
Unique Cities	10
Unique Companies	10

11. Data Quality Check

Missing-value analysis was performed using:

```
df.isnull().sum()
```

The result showed **0 missing values across all 15 columns**.

Therefore, the extracted dataset was complete and ready for basic analysis.

12. Basic Analysis

Users by City

There are **10 unique cities** for the 10 users.

Each city contains one user in this sample dataset.

Companies

There are **10 unique companies** associated with the 10 users.

Each company appears once.

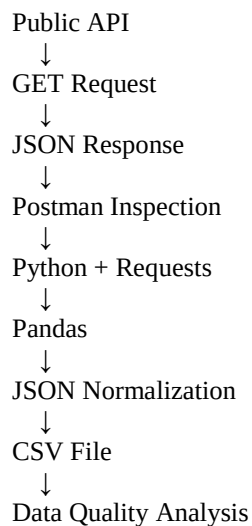
Data Structure

The original JSON contained nested address, geo, and company objects. These were successfully transformed into separate columns.

13. Key Findings

1. The API successfully returned **10 user records**.
 2. The JSON response contained **nested objects**.
 3. Pandas successfully flattened the nested JSON structure.
 4. The final dataset contained **10 rows and 15 columns**.
 5. There were **no missing values**.
 6. The data was successfully exported to **CSV format**.
 7. The resulting CSV can be used for further data analysis.
-

14. Final Data Flow



15. Conclusion

This assignment provided practical experience with REST APIs, HTTP GET requests, JSON data structures, nested JSON handling, Python requests, Pandas, and CSV conversion. The API data was successfully extracted, transformed into a structured dataset, exported to CSV, and analyzed for basic data quality and insights.

